

xv6-VMP I us

函数手册

中国海洋大学

2025年12月

目录

一、实验实现概述	3
二、设计方案与可行性分析	4
2.1 总体架构与缺页处理链	4
2.2 物理页管理：引用计数（kalloc 与 kref）	4
2.3 COW（Copy-On-Write，写时复制）	5
2.4 匿名 mmap/munmap	6
2.5 SAH：按 key 的共享匿名对象	9
2.6 进程生命周期集成	10
三、测试与结果分析	10
3.1 测试目的	11
3.2 测试方法	11
3.3 测试结果	11
3.4 结果分析	12
3.5 小结	13

一、实验实现概述

本实验在尽量保持 xv6 原有代码结构与接口风格的前提下进行扩展，主要改动点如下：

(1) kernel/kalloc.c: 引入物理页引用计数 kref (kref_get、kref_inc、kref_dec)，并改造 kfree() 与 freerange()，使共享页与 COW 页能够在正确时机安全回收。

(2) kernel/vm.c:

- a) 改造 uvmcopy(): fork 时不再复制物理页，而是共享映射并设置 COW 标记，减少复制与内存占用；
- b) 新增 cowbreak(): 在写缺页时按需拆分并复制物理页，恢复可写语义；
- c) copyout() 兼容 COW: 内核向用户地址写入前先处理 COW 页，避免绕过语义造成父子进程数据互相污染；
- d) 实现 vmfault(): 支持 sys_sbrk 的 lazy-heap (按需扩展堆)，降低一次性分配开销；
- e) 实现 vmafault(): 支持 mmap/SAH 的按需映射，将“记录元数据”与“实际分配/映射”解耦。

(3) kernel/trap.c: 在 usertrap() 中对 page fault 按优先级处理：

- a) 写缺页 (scause=15): cowbreak → vmafault(写) → vmfault
- b) 读缺页 (scause=13): vmafault(读) → vmfault

(4) kernel/sysproc.c: 实现 sys_mmap/sys_munmap/sys_shmctl, 包含 VMA 管理、自动选址、munmap 的 split 预检查与实际拆分逻辑；同时将 sys_sbrk 扩展为支持 eager/lazy 两种模式 (第二参数 t 控制)。

(5) kernel/shm.c / kernel/shm.h: 实现共享匿名对象表 (NSHM=16)，按 key 管理共享匿名页，支持 IPC_RMID 标记删除与引用计数回收。

(6) kernel/proc.h / kernel/proc.c: 在进程结构中新增 vmas[NVMA]; fork 时复制 VMA 并对共享 key 去重 shm_get; 释放阶段提供 vma_release_all() 与按 key 去重的 shm_put，避免泄漏与重复引用。

(7) `kernel/syscall.c`: 系统调用分发表中挂接 `SYS_mmap/SYS_munmap/SYS_shmctl` (以及 `sem` 相关接口, 为上层构建 `producer-consumer` 等同步模式预留)。

二、设计方案与可行性分析

2.1 总体架构与缺页处理链

本实验采用“元数据描述 (VMA) + 缺页按需分配 + 物理页引用计数”的整体架构, 在实现复杂度可控的前提下提升机制的可用性与稳定性。

设计方案

(1) 映射类需求 (`mmap/SAH/lazy-heap`) 在系统调用阶段仅记录元数据 (VMA 或 `p->sz`), 不立即分配物理页; 首次访问触发缺页后, 由 `fault handler` 统一完成分配与映射, 从而减少重复逻辑并避免一次性占用大量内存。

(2) 缺页处理链统一放在 `trap.c:usertrap()`:

- a) 写缺页 (`scause=15`): 优先尝试 COW (`cowbreak`), 再尝试 VMA 映射 (`vmafault`), 最后尝试 `lazy-heap` (`vmfault`);
- b) 读缺页 (`scause=13`): 先 `vmafault`, 再 `vmfault`;
- c) 三者都不处理则 `setkilled(p)`, 保持 `xv6` 一贯的“非法访问直接 `kill`”, 确保错误可观测且行为一致。

实现重点

- a) 写缺页必须先处理 COW, 否则会把“应当复制的共享只读页”误当成普通缺页或非法访问, 既破坏语义也会引入难排查的并发错误。
- b) 将 `mmap/SAH` 与 `sbrk-lazy` 统一走缺页路径, 可显著减少系统调用中的分配分支, 使边界条件 (如部分解除映射、退出回收) 更容易验证。

2.2 物理页管理: 引用计数 (`kalloc` 与 `kref`)

设计方案 (`kernel/kalloc.c`)

(1) 数据结构:

a) `kref.refcnt[MAXPAGES]`: 为每个物理页维护引用计数, 用于支撑共享页与 COW 页的安全释放。

b) `kref.lock`: 并发保护; 并提供三接口: `kref_get/kref_inc/kref_dec`。

(2) 分配/回收规则:

a) `kalloc()`: 成功分配物理页后将该页 `refcnt` 初始化为 1, 作为“被一个映射持有”的默认状态。

b) `kfree(pa)`: 不再无条件回收, 而是先 `kref_dec(pa)`:

- 1) 若结果 >0 : 说明仍被其他映射/对象引用, 直接返回, 不进入 `freelist`;
- 2) 若结果 $=0$: 才真正将页放回 `freelist`。

c) `freerange()`: 初始化阶段这些页并非由 `kalloc()` 得到, `refcnt` 初值为 0; 若直接 `kfree` 会出现负计数。故在初始化 `kfree(p)` 前先将该页 `refcnt` 置为 1, 再交由 `kfree` 正常进入 `freelist`。

实现重点

- a) COW/共享页的正确性强依赖 `kfree()` 的 `refcnt` 判定, 否则会出现“仍在使用但被回收”的致命错误, 直接导致随机崩溃或数据损坏。
- b) `freerange()` 初始化 `refcnt` 的处理是避免负计数的关键细节, 也是保证系统长期运行稳定性的前提。

2.3 COW (Copy-On-Write, 写时复制)

COW 的目标是优化 `fork`: 父子进程先共享物理页, 仅在写入时才复制, 适用于频繁 `fork` 的场景, 可显著减少内存拷贝与物理页分配压力。

设计方案 (`kernel/vm.c`)

(1) `fork` 共享阶段: `uvmcopy(old, new, sz)`

- a) 遍历父进程 $[0, sz)$ 的用户页表项：
 - 1) 若页表项不存在或无效直接跳过；
 - 2) 只对用户页 (PTE_U) 处理；
 - 3) 若该页原本可写 (PTE_W)：清写位并置 PTE_COW，父进程自身对应 PTE 也同步改为只读+COW；
- b) 子进程映射同一物理页地址 pa ，权限使用 (可能被改成) 只读+COW 的 flags；
- c) 每共享一页，都 $kref_inc(pa)$ 增加引用计数；
- d) 若 $mappages$ 失败，回滚：先 $kref_dec(pa)$ ，并对已建立的子进程映射 $uvmunmap(new, \dots, do_free=1)$ 。

(2) 写入拆分阶段： $cowbreak(pagetable, va)$

- a) 仅当页表项满足：有效、用户页、PTE_COW=1 且当前不可写时才处理；
- b) 若 $kref_get(old_pa)==1$ ：无需复制，直接恢复可写并清掉 PTE_COW；
- c) 否则：分配新页 $kalloc \rightarrow$ 拷贝旧页内容 $\rightarrow$ 对旧页 $kref_dec(old_pa) \rightarrow$ 更新 PTE 指向新页并恢复可写、清 COW $\rightarrow sfence_vma()$ 刷新 TLB。

(3) 内核写用户地址的兼容： $copyout()$

$copyout()$ 若写入目标页是 COW 页 (检测 PTE_COW)，会先调用 $cowbreak()$ ，保证内核写入不会绕过 COW 语义导致父子互相污染。

实现重点

- a) $uvmcopy()$ 必须同时修改父 PTE (降为只读+COW)，否则父进程仍可直接写共享页，破坏 COW。
- b) $cowbreak()$ 的 “ $refcnt==1$ 直接恢复可写” 能减少不必要的复制。
- c) $copyout()$ 兼容 COW 是必须项，否则用户态不写、但内核 $copyout$ 写入会造成共享污染或写失败。

2.4 匿名 $mmap/munmap$ ：可用的 VMA 管理、自动选址与安全拆分

该部分实现匿名映射与解除映射，核心是以 VMA 维护映射区间，并在缺页时按需分配/映射，使用户态可以获得“像真实系统一样可用”的大块内存映射能力。

2.4.1 VMA 数据结构与映射地址规划

数据结构 (kernel/proc.h)

struct vma { used, start, end, prot, flags, is_shm, shm_key }, 采用半开区间 $[start, end)$ 表示映射范围, 便于做重叠判断与拆分更新。

映射区规划 (kernel/sysproc.c)

- 1) MMAPBASE = 0x40000000 (从 1GB 开始, 避开低地址 heap, 减少与传统用户内存布局的冲突)
- 2) MMAPTOP = TRAPFRAME - 10*PGSIZE (为 trapframe 等固定映射预留余量, 避免越界覆盖)

2.4.2 sys_mmap: 严格校验、自动选址与 VMA 元数据写入

设计方案 (kernel/sysproc.c:sys_mmap)

(1) 参数校验:

- a) $len > 0$, $plen = \text{PGROUNDUP}(len)$ 非 0 且 $plen \leq (\text{MMAPTOP} - \text{MMAPBASE})$;
- b) prot 仅允许 PROT_READ/PROT_WRITE 组合: $(\text{prot} \ \& \ \sim(\text{PROT_READ}|\text{PROT_WRITE})) == 0$;
- c) flags 必须包含 MAP_ANON;
- d) addr 必须为 0 (本实现不支持用户指定地址)。

(2) MAP_SHARED 预处理 (在写入/占用 VMA 之前完成, 避免污染状态):

- a) $key \geq 0$, 并计算 $npages = plen / \text{PGSIZE}$;
- b) 若 $\text{shm_is_deleted}(key) == 1$ (已 IPC_RMID 标记), 拒绝新的 attach;
- c) 采用“按进程去重”的引用策略: 若当前进程还没有该 key 的其他 VMA ($\text{proc_has_shm_key}(p, key, 0) == 0$), 则设置 $\text{need_get} = 1$, 表示后续需要执行一次 $\text{shm_get}(key, npages)$ 。

(3) 分配 VMA 槽位并初始化:

- a) 通过 $\text{vma_alloc_slot}(p)$ 获取空槽, 若无空槽直接失败;
- b) 先 $\text{memset}(v, 0, \text{sizeof}(*v))$, 并将 $v \rightarrow \text{shm_key} = -1$, 保证失败路径不会残留脏状态。

(4) 自动选址 (first-fit):

- a) $\text{vma_find_space}(p, plen)$ 在 $[\text{MMAPBASE}, \text{MMAPTOP})$ 扫描空洞;
- b) 若发现重叠, 则跳到冲突 VMA 的 end 并 PGROUNDUP 后继续 (冲突末端由 $\text{vma_next_conflict_end}()$ 计算)。

(5) 写入 VMA 元数据 (仅记录, 不立即分配/映射物理页):

- a) 成功选址后填入 used/start/end/prot/flags;

b) 若 MAP_SHARED:

- 1) 若 need_get==1 则调用 shm_get(key, npages) (并记录 did_shm_get=1 便于回滚);
- 2) 设置 v->is_shm=1, v->shm_key=key, 后续缺页由 vmafault() 走共享对象路径。

(6) 失败回滚:

- a) 若已执行 shm_get(did_shm_get==1), 失败路径调用 shm_put(key) 回滚;
- b) 清空该 VMA 槽位关键字段 (used/is_shm/shm_key/start/end/prot/flags), 保证系统调用“要么成功, 要么不改变状态”。

2.4.3 sys_munmap: 支持局部解除映射的原子更新

设计方案 (kernel/sysproc.c:sys_munmap)

(1) 区间规整: 将 (uaddr, len) 规整为页边界 [a, b):

- a) a = PGROUNDDOWN(uaddr)
- b) b = PGROUNDUP(uaddr+len)
- c) 并检查溢出, 若 b<a 直接失败, 避免边界错误导致内核状态损坏。

(2) split 预检查 (保证一致性/原子性): 在真正修改任何 VMA 之前, 先确认拆分所需的槽位充足。

- a) 统计空闲 VMA 槽位 free_slots;
- b) 扫描所有与 [a, b) 重叠的 VMA, 统计“切在中间”需要 split 的次数 need_splits;
- c) 若 need_splits > free_slots, 直接返回 -1, 不做任何修改, 避免“做一半失败”破坏进程地址空间。

(3) 真正执行: 按重叠段逐段推进。

对每个与 [a, b) 重叠的 VMA, 计算重叠段 [seg_start, seg_end), 先:

- a) uvmunmap(p->pagetable, seg_start, (seg_end-seg_start)/PGSIZE, 1)
- b) 再更新 VMA, 按覆盖关系分四类:
- c) 覆盖整段: 删除该 VMA (vma_delete, 并按 key 去重决定是否 shm_put);
- d) 从头砍: start = seg_end;
- e) 从尾砍: end = seg_start;

- f) 中间砍：分裂为左右两段：左段收缩 `end=seg_start`，右段复制到新槽位并令 `start=seg_end`。

实现重点

- a) “先预检查 `split` 槽位，再执行”是 `munmap` 稳定性的关键：它用简单的前置条件保障了修改过程的原子性，从根源上避免地址空间被破坏。
- b) 解除页表映射与更新 `VMA` 分离处理，使回收顺序清晰（先撤销映射、再改元数据），也便于定位泄漏与 `double free`。

2.5 SAH：按 key 的共享匿名对象（零拷贝共享页池）

SAH 的核心是：在 `MAP_SHARED + key` 的语义下，让多个进程映射同一批匿名后备页，实现零拷贝共享，并支持按页 `lazy` 分配与 `IPC_RMID` 的完整删除语义。

设计方案（`kernel/shm.c`）

- (1) 共享对象表：固定槽位 `obj[NSHM]`，每项包含：

`used/key/npages/pa[]/refcnt/deleted`

其中 `pa[i]==0` 表示该页尚未分配，可在首次访问时再分配，降低大对象的内存占用。

- (2) 对象引用管理：

- a) `shm_get(key, npages)`：查找或创建对象；成功则对象 `refcnt++`（新建为 1）。若 `deleted==1` 或 `npages` 超过已有对象限制则失败，保证对象参数一致。
- b) `shm_put(key)`：`refcnt--`；当 `refcnt==0` 时释放对象所有已分配页（逐页 `kfree`）并回收槽位，实现延迟回收。

- (3) 按页 `lazy` 分配：

`shm_getpa(key, page_index)`：若 `pa[page_index]==0` 则 `kalloc()+memset(0)` 并缓存；否则直接返回已有 `pa`，保证多进程看到同一物理页。

- (4) `IPC_RMID`：

`shm_ctl(key, IPC_RMID)`：设置 `deleted=1`，拒绝后续新的 `shm_get`；若此时 `refcnt==0` 可立即释放并回收槽位，否则等待最后一次 `shm_put` 再释放。

实现重点

- a) “标记删除 + 延迟回收”保证：已 attach 的进程仍可继续使用；同时新 attach 被拒绝，符合真实 IPC_RMID 语义。
- b) `pa[i]==0` 才分配的策略使共享对象可以定义得很大，但只为实际访问的部分付出物理内存成本，体现按需分配的价值。

2.6 进程生命周期集成：fork / 释放阶段与资源回收

该部分将 VMA 与共享对象引用纳入进程生命周期管理，确保 `fork`、`munmap`、`exit` 等路径下不泄漏、不重复增减引用，并避免释放顺序错误导致的崩溃。

设计方案（`kernel/proc.c`）

(1) `fork`：复制 VMA 元数据并对共享 key 去重 `shm_get`

- a) 子进程通过 `memmove(np->vmass, p->vmass, ...)` 继承 VMA；
- b) 遍历子进程 VMA，若是共享映射则对 key 去重后调用 `shm_get(key, npages)`；若失败则回滚释放子进程资源，保证 `fork` 的失败路径也无泄漏。

(2) 释放阶段：VMA 解除映射 + `shm_put` 去重

`vma_release_all(p)`：遍历所有 VMA，解除页表映射；随后按 key 去重 `shm_put`；最后清空 VMA 元数据。

（注：本实现对同一 key 的多段 VMA 采取“按 key 去重 `shm_put`”的方式归还对象引用，避免 `refcnt` 被重复扣减。）

实现重点

- a) 对 key 去重是共享对象 `refcnt` 正确性的核心：若同一进程存在多个相同 key 的 VMA，必须保证引用只增减一次，否则会造成过早释放或永不释放。
- b) `fork`/`munmap`/`exit` 三条路径都要严格遵循“先撤销页表映射，再更新元数据，最后更新引用计数”的顺序，以保证可行性与可维护性。

三、测试与结果分析

3.1 测试目的

本次测试旨在对比两种进程间通信（IPC）机制在**大数据量传输场景**下的性能与开销差异：

- 1) **PIPE IPC**: 典型的内核缓冲区转发模型, 数据需要在用户态与内核态之间多次拷贝。
- 2) **SHM + Semaphore IP**: 生产者/消费者共享同一段内存, 通过信号量同步, 目标是尽量消除数据拷贝开销。

通过基准程序输出的时间、内存分配、拷贝字节数、缺页统计等指标, 验证共享内存设计在吞吐场景下的优势, 并观察其一次性初始化成本(缺页、分配次数等)。

3.2 测试方法

3.2.1 测试程序与运行方式

使用基准测试程序 `ipc_bench`, 在相同环境下分别执行 PIPE 与 SHM+SEM 两种模式测试。运行命令如下:

```
$ ipc_bench
```

测试负载

两组测试传输的数据规模一致, 约为 **8MB**。

指标说明

- 1) **time (ticks)**: 完成一次测试所消耗的时钟 tick 数 (越小越好)。
- 2) **kalloc**: 测试过程中触发的内存页分配次数 (反映内存管理/初始化开销)。
- 3) **copyin_bytes / copyout_bytes**: 用户态与内核态之间发生的数据拷贝量 (拷贝越多通常越慢)。
- 4) **faults: cow / lazy / shm**
 - a) **cow**: Copy-on-Write 相关缺页 (通常发生在 fork 后首次写入等场景)。
 - b) **lazy**: 延迟分配导致的缺页 (首次触发才真正分配/映射)。
 - c) **shm**: 共享内存相关缺页 (首次访问共享页时建立映射/分配等)。

3.3 测试结果

原始数据

PIPE IPC:

- 1) time: **353 ticks**
- 2) kalloc: **8**
- 3) copyin_bytes: **8,388,608**

- 4) copyout_bytes: 8,388,664
- 5) faults: cow=1 lazy=0 shm=0

SHM + SEM IPC:

- 1) time: 138 ticks
- 2) kalloc: 28
- 3) copyin_bytes: 0
- 4) copyout_bytes: 48
- 5) faults: cow=1 lazy=33 shm=33

对比汇总表

指标	PIPE IPC	SHM + SEM IPC	变化/结论
传输数据量	~8MB	~8MB	一致
time (ticks)	353	138	降低到约 39% ($\approx 2.56\times$ 更快)
copyin_bytes	8,388,608	0	拷贝基本消除
copyout_bytes	8,388,664	48	仅剩极小控制开销
kalloc	8	28	SHM 初始化/映射带来更多分配
faults	cow=1, lazy=0, shm=0	cow=1, lazy=33, shm=33	SHM 首次访问触发缺页属预期

说明： $353/138 \approx 2.56$ ，因此共享内存方案在该负载下整体耗时约为 PIPE 的 39%，性能提升显著。

3.4 结果分析

(1) 为什么 PIPE 更慢：核心瓶颈在“数据拷贝路径”

PIPE 模式下，8MB 数据需要通过内核缓冲区中转，通常表现为：

- 1) 发送方：用户态 $\rightarrow$ 内核态 (copyin)
- 2) 接收方：内核态 $\rightarrow$ 用户态 (copyout)

因此出现 $\text{copyin_bytes} \approx 8\text{MB}$ 且 $\text{copyout_bytes} \approx 8\text{MB}$ 的结果，说明**整段数据几乎完整经历了两次跨域拷贝**。这类拷贝在大数据量下会成为主要耗时来源，从而导致 $\text{time}=353$ ticks。

(2) 为什么 SHM + SEM 更快：数据“零拷贝”传输

共享内存方案中，生产者与消费者在用户态直接读写同一段共享页，信号量只负责同步，不需要内核缓冲区搬运数据。因此：

- 1) copyin_bytes=0
- 2) copyout_bytes=48

拷贝路径基本被消除，使得总耗时下降到 138 ticks，带来约 $2.56\times$ 的提升。

(3) 共享内存的“第一次成本”：缺页与分配次数上升是合理现象

SHM + SEM 的输出中：

- kalloc=28 明显高于 PIPE 的 8
- lazy=33, shm=33 也明显更多

这说明共享内存存在首次建立/首次访问时存在一次性成本：例如共享页的延迟分配、首次触发映射建立等。该成本集中体现在“第一次触发缺页”，属于共享内存机制的典型特征；但对大数据量传输而言，一次性成本远小于反复数据拷贝的代价，因此总体仍显著更快。

3.5 小结

在同等 8MB 负载下，PIPE IPC 因为需要完成大规模 copyin/copyout，耗时为 **353 ticks**；而 SHM + SEM IPC 通过共享页直接读写，将数据拷贝量降至几乎为零（copyin=0, copyout=48），总耗时降至 **138 ticks**，达到约 $2.56\times$ 的性能提升。尽管共享内存方案在首次访问时带来更多 kalloc 与 lazy/shm faults，但这属于可接受的一次性初始化成本，整体验证了共享内存机制在大数据量 IPC 场景下的显著优势。